Filters from the words

Structured output: asking a model for data instead of prose

Your Step 1 search filters correctly, but only when the filters arrive from the chips and the price menu. A customer who types “under 4000” gets nothing filtered. In this step you add one function that reads the filters out of the words, before the search runs. Your `search` and `filter_products` do not change.

contents

1 The problem **2**

2 A model that fills in a form **2**

3 The instruction **3**

4 Clicks and words together **4**

5 Where it plugs in **4**

6 The exercise **5**

7 Experiments, after it passes **5**

1 The problem

Type this in the search box, with the price menu on *Any*:

> *A very comfortable bed under 4000*

These are the five results your Step 1 search returns:

Product	Price	Score
LYNGSBY mattress	1 999 kr	0.665
LYNGESKOV mattress	8 969 kr	0.656
ASKSLØSE mattress	2 119 kr	0.653
RINGSGAARD mattress	3 459 kr	0.653
STRANDABJERG mattress	8 919 kr	0.653

Table 1 "A very comfortable bed under 4000" with no menu choice. Two of five are far above the price the customer gave.

Your filter is not broken. It only removes products when `filters.max_price` has a value, and with the menu on *Any* it is `None`. The words "under 4000" never reach it. They go into the embedding, and an embedding cannot compare numbers: to it, "under 4000" is a few words with a vague money feel.

The fix is to read the sentence *before* the search, and split it into the part that is meaning and the part that is a rule:

```
"A very comfortable bed under 4000"
   search_text = "A very comfortable bed"     -> the embedding
   category    = "mattress"                   -> filter_products
   max_price   = 4000                         -> filter_products
```

2 A model that fills in a form

This job needs the *text model*, not the embedding model. The embedding model only ever returns numbers. The text model, `gemini-3-flash-preview`, reads the sentence and can write an answer.

Left alone, it writes prose: "Sure! It looks like you are looking for a mattress under 4000 kroner…". Your code cannot use that. What it needs is data: three fields with exact values. This is what *structured output* gives you. You hand the model a form, and it must fill in exactly that form.

structured output: the model must reply with data in a shape you define, not free text.

The form is a Python class:

```python
class ParsedQuery(BaseModel):
    search_text: str
    category: Category | None     # one of the 8, or None
    max_price: float | None       # a number, or None
```

It is the same kind of class as `SearchFilters`: a *Pydantic model*. Here it does a second job: it is turned into a *JSON schema* (a description of the exact shape a piece of JSON must have) that the model must follow. Because `category` is a `Category`, the model can only answer one of the eight category names, or `null`. It cannot invent "bed".

Pydantic model: a class that describes fields and their types, and checks values against them.

The call has three settings that matter:

```
config = types.GenerateContentConfig(
    system_instruction=INSTRUCTION,          # what to do
    response_mime_type="application/json",   # reply with JSON
    response_schema=ParsedQuery,             # in exactly this shape
)
reply = client.models.generate_content(
    model=TEXT_MODEL, contents=text, config=config)
reply.parsed     # a ready ParsedQuery, not text
```

The schema fixes the *shape* of the answer. Only the instruction fixes its *meaning*: when to set a field, and when to leave it empty.

3 The instruction

The *system instruction* is the only place where you tell the model what you mean. The schema says "`max_price` is a number"; it does not say that "180x200" is a size and not a price. The instruction has to.

system instruction: the text that tells the model its job. It is sent with every request, before the customer's words.

The test sentences in `data/eval/filter_queries.json` contain the traps worth writing down:

Sentence	Category	Price	The trap
billig madras under 2000 kr	mattress	2000	Danish words
dining table for 8 people, budget 6000	dining_table	6000	the 8 is a count
mattress 180x200	mattress	none	a size, not a price
sofa between 2000 and 5000 kr	sofa	5000	a range: the upper end
something cosy for the living room floor	none	none	no category is said

Table 2 Five of the fifteen test sentences and what makes each hard.

The last row is the important one. In the notes, a first instruction that only said "only set a filter if the customer clearly says it" still made the model answer *rug*. A filter is a hard cut: a wrong category hides every other product, however good. So the instruction must say plainly that a missing category is `null`, and that guessing is wrong. An empty field costs nothing; the embedding still finds rugs by meaning.

4 Clicks and words together

A customer can type a price *and* pick one in the menu. Then there are two answers, and one must win. The rule here is simple: **a click wins**, because it is the more deliberate choice.

Typed "bed under 4000", and clicked	category	max_price	from
nothing	mattress	4000	both from the words
the price menu at 2 500	mattress	2500	price from the click
the Sofa chip	sofa	4000	category from the click

Table 3 What `merge_filters` returns. Each field is decided on its own.

5 Where it plugs in

The API now does one thing before calling your search (`read_filters` in `api/app.py`):

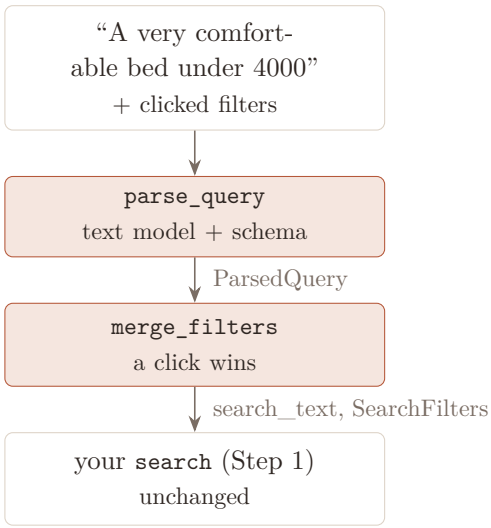

Figure 1 The two highlighted steps are this exercise.

While `parse_query` still raises `NotImplementedError`, or if the text model fails, the API skips it and searches exactly as in Step 1. A search never breaks because of this step.

When it works, the results line on the page says what was read, for example *filtered by your words: Mattress, up to 4.000 kr.*

The cost. Each new search makes one extra call to the text model: roughly half a second to a second and a half, and a tiny fraction of an øre. `@lru_cache` on `parse_query` remembers each text, so "Show more" does not ask again.

6 The exercise

Everything is in `api/parse_query.py`. Work from top to bottom:

1. **TODO 1**: add `category` and `max_price` to `ParsedQuery`, with no default value, so the model must always fill them.
2. **TODO 2**: `merge_filters`, the table in section 4. It is plain Python; Part 1 of the check tests it without calling Gemini.
3. **TODO 3**: write `INSTRUCTION`. Use section 3 and the fifteen sentences.
4. **TODO 4**: `parse_query`, the call in section 2.

Check your work from the repo root:

```
python3 steps/01b_filters_from_words/check.py
```

- **Part 1** tests the class and `merge_filters`.
- **Part 2** reads the fifteen sentences. The category and the price must each be right in at least 90 % of them. Every miss prints what was wanted and why.
- **Part 3** runs your Step 1 search on its 23 questions, without and with the filters. “sofa under 4000 kr” should go from precision 0.00 to 1.00, and the graded questions must not get worse.

7 Experiments, after it passes

- Remove the “do not guess” sentence from your instruction and run the check. Which sentences change?
- Search “something cosy for the living room floor” in the shop, with and without your instruction’s rule. Count the products that can still appear.
- Try “a sofa for less than three thousand”. Numbers in words are a new trap.
- “sofa that is not leather” and “mattress 180x200” cannot be filtered yet: `SearchFilters` has no field for material or size. What would you add, and where?